



DM-TacClaw 视触觉夹爪

产品说明书



版本: V1.0 20260529

1. 产品概况	1
1.1 夹爪功能亮点	1
1.2 应用场景	1
1.3 夹爪规格参数	2
1.4 夹爪尺寸、模型与坐标规定	4
1.5 包装清单	5
2. 夹爪安装与接线	5
3. 夹爪软件说明	5
3.1 软件及系统环境要求	5
3.2 硬件要求	6
3.3 系统安装	6
3.4 视触觉夹爪 API 使用说明	6
3.4.1 鱼眼相机 API	6
3.4.2 夹爪运动控制 API	19
3.4.3 视触觉传感器 API	23
4. 注意事项	23
4.1 夹爪维护与保养	23
4.1.1 日常清洁	23
4.1.2 定期检查、护理	24
4.2 传感器维护与保养	24
4.2.1 定期检查与清洁	24
4.2.2 温度与湿度控制	25
4.2.3 定期功能测试	25
4.2.4 防止污染与灰尘	25
4.2.5 避免物理损伤	25
4.2.6 线束的更换和感知层的更换	25

1. 产品概况

DM-TacClaw 视触觉夹爪是戴盟推出的新一代智能数据采集夹爪，实现了多模态感知与系统同构能力深度融合，不仅具备大行程与高精度夹持控制的优势，更将广角鱼眼相机集成于夹爪本体，二指内置高分辨率视触觉传感器，实现视觉与触觉的协同感知，精准捕捉环境信息与接触细节，让数据采集过程具备更丰富的感知维度与更强的一致性。

凭借系统同构与灵活控制的协同优势，DM-TacClaw 在相机视角下的空间构型与无本体数采夹爪 DM-DataClaw 保持一致，支持模型的快速迁移与验证；同时开放位置、速度、力矩等控制接口，实现高灵活度的运动控制与策略适配，即使在多类型物体的复杂抓取任务中，也能始终保持稳定与精确的操作表现。该产品具备 100mm 大行程与 0.05mm 定位精度，兼顾大范围抓取与精细操作需求。

DM-TacClaw 兼容多种控制协议，支持灵活参数配置与部署，可广泛应用于工业检测、精密装配、智能物流、人机交互等多个领域，为各类高稳定性和高灵活度的自动化数据采集与执行场景提供强有力的技术支持。

1.1 夹爪功能亮点

- 多模态感知一体化：内置广角鱼眼相机覆盖操作视野，二指集成高分辨率视触觉传感器，实现视觉与触觉的深度融合感知，精准捕捉环境信息与接触细节。
- 性能与适用性兼顾：具备 100mm 大行程与 0.05mm 定位精度，兼顾大范围抓取与精细操作需求，适用于多类型物体的稳定夹持与复杂任务执行。
- 系统同构与灵活控制：鱼眼视角下的空间构型与无本体数采夹爪保持一致，支持模型快速迁移与验证；同时开放位置、速度、力矩等参数配置，实现高灵活度的运动控制与策略适配。

1.2 应用场景

DM-TacClaw 视触觉夹爪集成视觉、视触觉感知与夹爪控制能力，既可作为数据采集与执行平台，用于多模态数据采集、模型验证、模型迁移和推理实现，也可作为高分辨率视触觉夹爪，应用于多类型物体的抓取、搬运、装配和交互等任务场景。主要应用场景包括：

1. 数据采集与任务测试

适用于复杂抓取任务中的多模态数据采集与执行测试。产品能够在夹取、接触、移动和操作过程中同步获取环境信息与接触细节，适用于抓取状态识别、物体姿态判断、

接触反馈分析、操作策略优化等任务，为后续模型训练、算法优化和应用落地提供数据基础。

2. 模型验证与推理实现

适用于机器人模型在真实操作环境中的验证与部署。作为与无本体数采夹爪 DM-DataClaw 同构的执行器，产品可用于模型推理的真机部署：通过实时输入视觉、触觉和运动信息，对模型的抓取效果、接触判断、操作策略和推理控制结果进行测试与评估，并辅助完成训练模型到实际操作流程中的迁移与验证。

3. 工业制造与加工

适用于机床上下料、工件抓取与搬运、精密装配、工业检测等场景。通过视触觉感知与高精度夹持控制，产品可帮助机器人在接触类操作中获取丰富的感知信息，提高抓取与装配过程的稳定性。

4. 智能物流与包装

适用于包装抓取与分拣、智能仓储、自动化配送等场景。产品可面向不同尺寸、形状和材质的物体进行抓取与搬运，满足物流分拣和柔性包装和搬运任务中对稳定夹持和灵活控制的需求。

5. 科研与实验室

适用于机器人操作控制、视触觉感知、多模态模型、数据采集平台搭建等科研与教学场景。用户可基于产品开展模型训练、算法验证、实验演示和机器人教育等工作。

6. 商业与服务

适用于新零售自动化、人机交互及服务机器人等场景。产品可为开放环境中的抓取、交互和操作任务提供视触觉感知支持，提升机器人面对多样化物体和复杂操作需求时的适应能力。

1.3 夹爪规格参数

表 1-1 夹爪参数表

型号	DM-TacClaw
尺寸	194.5 mm*124.5 mm*177 mm
重量	1010g
夹爪结构	角度式二指

开合行程和角度	0-100mm; 0-35°
夹持力	4N~24N
视觉系统	鱼眼相机 1920*1080 60fps
最大推荐负载	2kg
重复定位精度	0.05mm
传动方式	齿轮传动
输出数据类型	Protobuf
数据传输接口	以太网口
数据传输方式	gRPC+UDP 通讯
电源接口	XT30 母头
工作电压	24V-48V
额定电流	0.55A
峰值电流	2A
推荐使用环境	0~40°C, 85% RH 以下

1.4 夹爪尺寸、模型与坐标规定

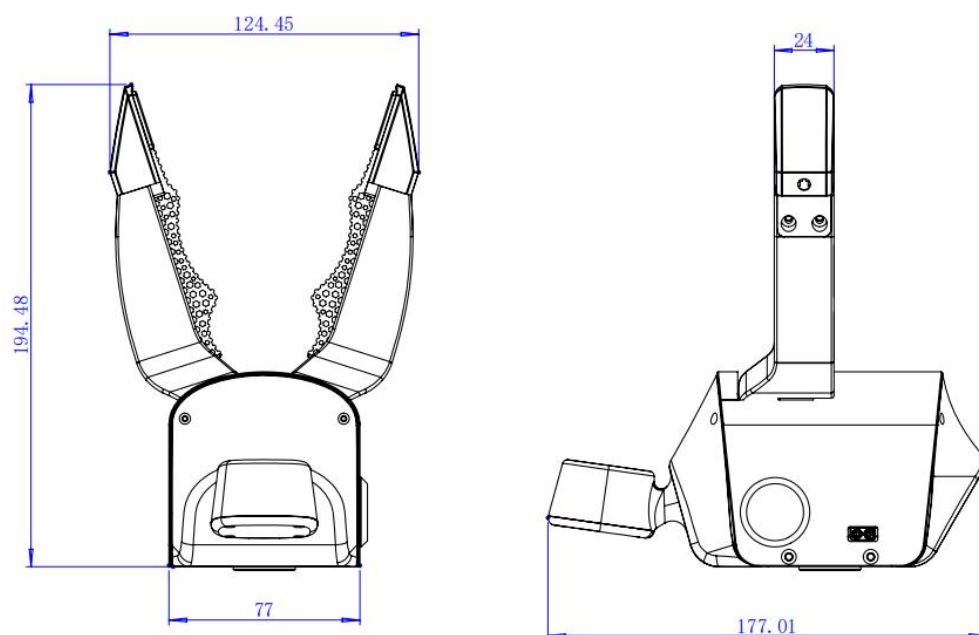


图 1.1 夹爪尺寸图 (单位:mm)

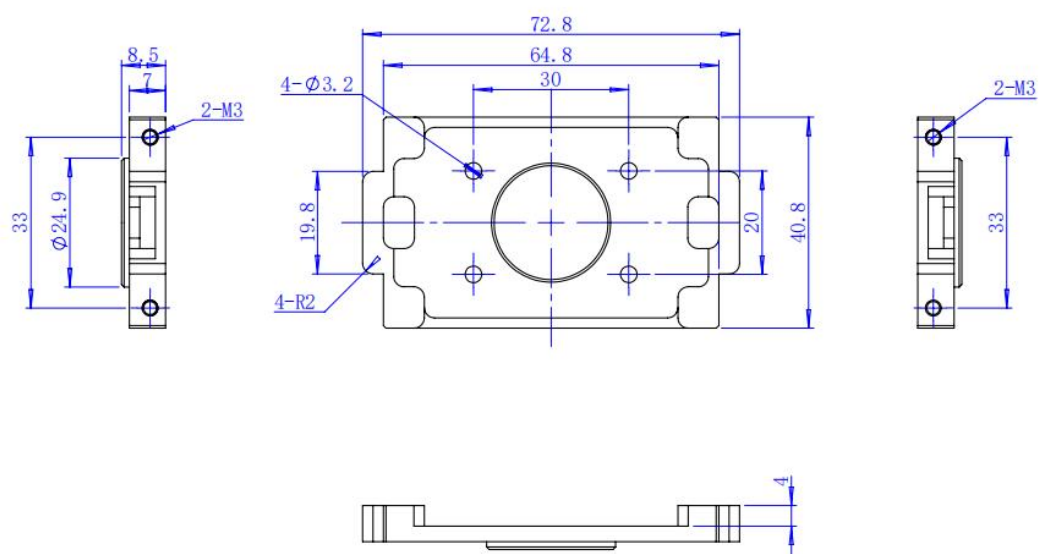


图 1.2 安装尺寸图 (单位: mm)

1.5 包装清单

表 1-2 DM-TacClaw 包装清单

产品	数量	备注
视触觉夹爪	1	已集成视触觉传感器与连接件
U 盘	1	

2. 夹爪安装与接线

本产品通过外部电源供电，并通过网线与电脑连接，参考图 2.1。连接时，请先将外部电源线接入黄色电源接口，为视触觉夹爪供电；随后掀开黑色以太网接口外的保护罩，将 CAT6 网线插入该接口，并将网线另一端连接至电脑。连接完成后，视触觉夹爪即可与电脑进行通信。

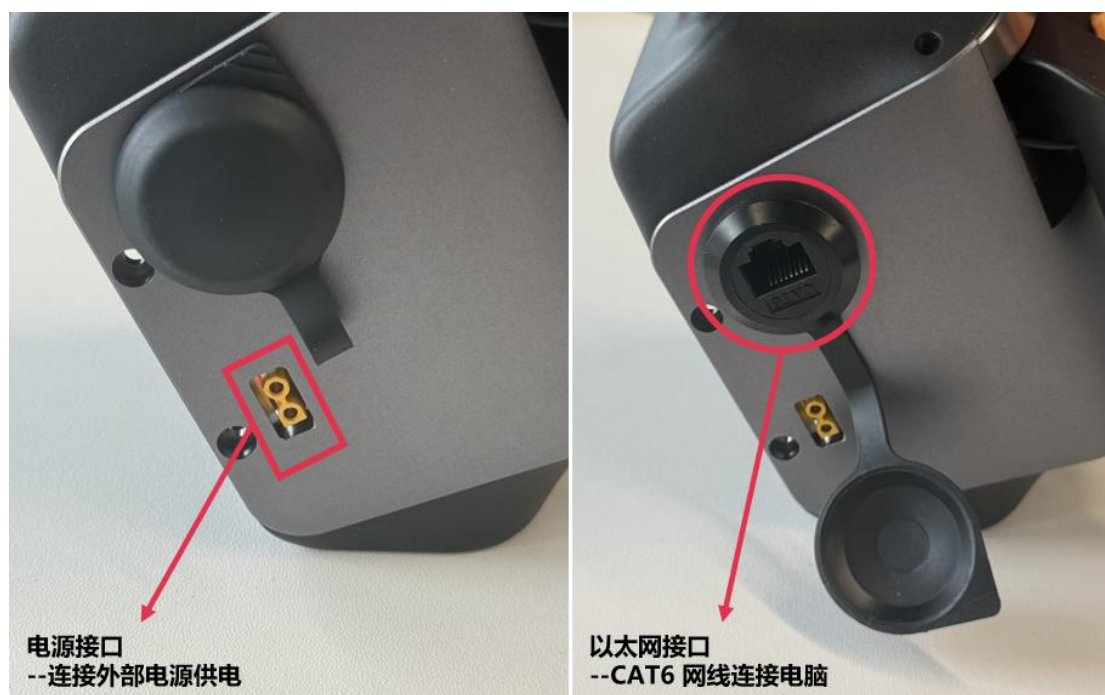


图 2.1 接口示意

3. 夹爪软件说明

3.1 软件及系统环境要求

1. 操作系统:

Linux / Windows x86

2. Python 版本:

Python 3.8 ~ 3.10 (不包括 3.11 及以上)

3. 依赖库:

grpcio >= 1.76.0

grpcio-tools >= 1.76.0

protobuf >= 6.33.2

pyserial >= 3.5

3.2 硬件要求

1. 处理器:

- 支持 x86_64 架构的处理器 (推荐使用多核处理器以提高性能)。

2. 内存:

- 最低要求: 8GB RAM。

3. 存储:

- 至少需要 1GB 的可用磁盘空间, 用于安装软件及其依赖项。

4. 显卡:

- 支持 CUDA 的 NVIDIA 显卡 (需要安装对应的 CUDA 驱动程序, 以支持 cupy 库, 默认 CUDA 12)。

5. 其他:

- 需要千兆网线以及本地主机需要有对应网口。

3.3 系统安装

在计算机上安装 Ubuntu/Windows 系统。

3.4 视触觉夹爪 API 使用说明

3.4.1 鱼眼相机 API

鱼眼相机使用 RemoteCameraCapture 远程相机客户端；

remote_camera.py 提供 RemoteCameraCapture 类，用于从远端相机服务获取实时图像。该类采用 OpenCV 风格接口，控制链路通过 gRPC 建立，图像数据通过 UDP 接收。

- 文件路径：fish_cam/remote_camera.py
- 主要类：RemoteCameraCapture
- 支持编码：HEVC、MJPG
- 图像格式：返回 numpy.ndarray，颜色通道为 OpenCV 常用的 BGR，形状通常为 (height, width, 3)

基本使用流程

- 功能：创建远程相机对象，打开连接，循环读取图像，程序结束前释放资源。
- 示例：

```
Python
from fish_cam.remote_camera import RemoteCameraCapture

cap = RemoteCameraCapture(
    host="192.168.127.10",
    port=50088,
    codec="HEVC",
    width=1280,
    height=720,
    fps=60,
)

try:
    cap.open()

    ok, frame = cap.read(timeout=1.0)
    if ok:
        print("获取图像成功: ", frame.shape)
    else:
        print("读取超时或暂未收到图像。")
finally:
    cap.release()
```

3.4.1.1 RemoteCameraCapture()

- 功能：创建远程相机客户端对象。构造函数只保存连接参数和采集参数，不会立即连接服务端；实际连接在 `open()` 或首次 `read()` 时建立。

- 参数：

参数	类型	默认值	说明
<code>host</code>	<code>str</code>	必填	远端相机服务 IP 或主机名。
<code>port</code>	<code>int</code>	50088	远端相机服务 gRPC 端口。
<code>codec</code>	<code>str</code>	"HEVC"	图像编码格式，支持 "HEVC"、"MJPG"。
<code>width</code>	<code>int</code>	1280	请求图像宽度。服务端可能根据实际能力返回协商后的宽度。
<code>height</code>	<code>int</code>	720	请求图像高度。服务端可能根据实际能力返回协商后的高度。
<code>fps</code>	<code>int</code>	60	请求帧率。服务端可能根据实际能力返回协商后的帧率。
<code>window_name</code>	<code>str</code>	"Remote Camera"	<code>show()</code> 显示窗口名称。
<code>ffmpeg_bin</code>	<code>str</code>	"ffmpeg"	HEVC 解码使用的 ffmpeg 可执行文件名称或路径。
<code>udp_port</code>	<code>int</code>	0	本地接收 UDP 图像数据的端口。0 表示由系统自动选择。
<code>bind_host</code>	<code>str</code>	"0.0.0.0"	本地 UDP 监听地址，一般保持默认即可。
<code>client_ip</code>	<code>str</code>	""	指定服务端回传 UDP 数据时使用的客户端 IP。多网卡或跨网段场景建议设置。
<code>device</code>	<code>str</code>	""	指定远端相机设备；一般保持默

			认即可。
<code>max_datagram</code>	<code>int</code>	<code>1200</code>	单个 UDP 数据包最大长度。一般保持默认即可。
<code>reassembly_timeout</code>	<code>float</code>	<code>0.5</code>	MJPEG UDP 分片重组超时时间，单位为秒。

- 返回值：返回 `RemoteCameraCapture` 对象。
- 示例：

Python

```
from fish_cam.remote_camera import RemoteCameraCapture

cap = RemoteCameraCapture(
    host="192.168.127.10",
    codec="MJPG",
    width=1920,
    height=1080,
    fps=30,
    client_ip="192.168.127.20",
)
```

3.4.1.2 list_capabilities()

- 功能：查询远端相机服务支持的设备、编码格式、分辨率和帧率。该方法是类方法，不需要先创建 `RemoteCameraCapture` 对象。
- 参数：

参数	类型	默认值	说明
<code>host</code>	<code>str</code>	必填	远端主机名。
<code>port</code>	<code>int</code>	<code>50088</code>	远端相机服务 gRPC 端口。
<code>device</code>	<code>str</code>	<code>""</code>	指定查询某个设备；为空表示查询服务端可用设备。

- 返回值：返回 gRPC 的 `CapabilityResponse` 对象。常用字段包括：

字段	说明
<code>response.cameras</code>	相机设备列表。
<code>camera.device</code>	设备名称。
<code>camera.codecs.codec</code>	编码格式，例如 HEVC 或 MJPG。
<code>codec.mode.width / codec.mode.height</code>	支持的分辨率。
<code>mode.fps</code>	支持的帧率列表。

- 异常：连接失败或服务端异常时，可能抛出 `grpc.RpcError`。
- 示例：

Python

```
from fish_cam.remote_camera import RemoteCameraCapture

response = RemoteCameraCapture.list_capabilities("192.168.127.10",
port=50088)

for camera in response.cameras:
    print("设备: ", camera.device)
    for codec in camera.codecs:
        print("  编码: ", codec.codec)
        for mode in codec.modes:
            print("    分辨率: {}x{}, 帧率: {}".format(
                mode.width,
                mode.height,
                list(mode.fps),
            ))
```

3.4.1.3 open()

- 功能：建立与远端相机服务的连接，启动控制流和图像接收。所有稳定的取图操作建议在 `open()` 成功后进行。
- 行为说明：

RemoteCameraCapture 编码	行为
HEVC	启动 ffmpeg，通过 UDP 接收 MPEG-TS 数据并解码为 BGR 图像。
MJPEG	监听本地 UDP 端口，接收 MJPG 分片，重组后使用 OpenCV 解码。

- 返回值：连接成功返回 `True`。如果对象已经打开，再次调用也返回 `True`。
- 异常：连接失败、编码不支持、服务端返回错误、ffmpeg 启动失败或 OpenCV 未安装时，抛出异常。
- 示例：

```
Python
cap = RemoteCameraCapture(host="192.168.127.10", codec="HEVC")

try:
    if cap.open():
        print("相机连接成功。")
except Exception as exc:
    print("相机连接失败：", exc)
finally:
    cap.release()
```

3.4.1.4 isOpened()

- 功能：判断远程相机当前是否处于可读取状态。
- 返回值：`True` 表示当前连接和接收链路可用；`False` 表示连接未打开、已关闭或接收链路已异常退出。
- 示例：

```
Python
cap.open()

if cap.isOpened():
    print("相机已打开，可以读取图像。")
else:
```

```
print("相机未打开或连接已断开。")
```

3.4.1.5 read()

- 功能：读取一帧最新图像。
- 参数：

参数	类型	默认值	说明
<code>timeout</code>	<code>float</code> 或 <code>None</code>	<code>None</code>	等待图像的最长时间，单位为秒。 <code>None</code> 表示一直等待，直到收到图像或连接异常。

- 返回值：

返回值	说明
<code>ok</code>	<code>True</code> 表示读取到图像， <code>False</code> 表示超时、连接关闭或暂未收到图像。
<code>frame</code>	读取成功时为 BGR 格式的 <code>numpy.ndarray</code> ；读取失败时为 <code>None</code> 。

- 行为说明：
- 如果尚未调用 `open()`，`read()` 会自动尝试打开连接。
- `timeout` 建议设置为有限值，例如 `1.0`，避免业务线程长期阻塞。
- 返回的 `frame` 是当前帧的拷贝，业务代码可以安全处理。
- 使用 HEVC 时，如果 `ffmpeg` 解码进程异常退出，可能抛出 `RuntimeError`。
- 示例：

```
Python
cap.open()

while cap.isOpened():
    ok, frame = cap.read(timeout=1.0)
    if not ok:
        error = cap.get("error")
        if error:
            print("读取失败：", error)
        continue
```

```
print("收到图像: ", frame.shape)
```

3.4.1.6 get_intrinsics()

- 功能：读取远端相机的内参、相机内参矩阵和畸变系数。
- 建议：在 `open()` 成功后调用，确保读取的是当前实际打开的相机。
- 返回值：

```
Python
{
    "intrinsics": [fx, fy, cx, cy],
    "camera_matrix": [
        [fx, 0.0, cx],
        [0.0, fy, cy],
        [0.0, 0.0, 1.0],
    ],
    "distortion_coeffs": [k1, k2, k3, k4],
}
```

- 说明：服务端会缓存标定数据。第一次读取可能较慢，后续读取会直接返回缓存。
- 异常：远端服务未启动、连接超时、相机不存在或相机 flash 中无有效标定数据时会抛出异常。

示例：

```
Python
cap = RemoteCameraCapture(host="192.168.127.10", codec="HEVC")

try:
    cap.open()
    data = cap.get_intrinsics(timeout=5.0)
    print("内参:", data["intrinsics"])
    print("相机矩阵:", data["camera_matrix"])
    print("畸变系数:", data["distortion_coeffs"])
finally:
    cap.release()
```

3.4.1.7 get_sn()

- 功能：读取远端相机的 SN 号。

- 建议：在 `open()` 成功后调用，确保读取的是当前实际打开的相机。
- 返回值：字符串形式的 SN 号。

```
Python
"FE20260520010001"
```

- 说明：`get_sn()` 与 `get_intrinsics()` 使用同一份服务端标定缓存，不会在缓存命中时重复读取相机 flash。
- 异常：远端服务未启动、连接超时、相机不存在或相机 flash 中无有效标定数据时会抛出异常。

示例：

```
Python
cap = RemoteCameraCapture(host="192.168.127.10", codec="HEVC")

try:
    cap.open()
    sn = cap.get_sn(timeout=5.0)
    print("相机 SN 号:", sn)
finally:
    cap.release()
```

3.4.1.8 `get_latest_frame()`

- 功能：获取客户端当前缓存的最新一帧图像。
- 参数：无。
- 返回值：

返回值	类型	说明
frame	<code>numpy.ndarray</code> 或 <code>None</code>	如果客户端已经收到过图像，则返回最新一帧 BGR 格式图像；如果尚未收到图像，则返回 <code>None</code> 。

- 行为说明：
 - `get_latest_frame()` 不会等待新图像，只会立即返回当前缓存中的最新帧。
 - 该函数同时支持 `HEVC` 和 `MJPEG` 两种编码方式，用户无需根据编码方式分别处理。

- 返回的 `frame` 是缓存帧的拷贝，业务代码可以安全处理。
- 示例：

```
Python
frame = cap.get_latest_frame()

if frame is None:
    print("当前还没有收到图像")
else:
    print("最新图像尺寸：", frame.shape)
```

3.4.1.9 `get(name: str)`

- 功能：获取远程相机的状态信息、统计信息或错误信息。
- 参数：

参数	类型	默认值	说明
name	<code>str</code>	无	要查询的状态字段名称。

- 常用字段：

Name	返回值说明
<code>"width"</code>	当前图像宽度。
<code>"height"</code>	当前图像高度。
<code>"fps"</code>	当前帧率。
<code>"codec"</code>	当前编码方式，取值为 <code>"HEVC"</code> 或 <code>"MJPG"</code> 。
<code>"frame_id"</code>	当前客户端收到的帧编号。
<code>"device"</code>	服务端实际使用的相机设备，例如 <code>/dev/video4</code> 。
<code>"session_id"</code>	当前远程相机会话 ID。
<code>"decoded_frames"</code>	客户端已解码的帧数。

"returned_frames"	通过 <code>read()</code> 返回给用户的帧数。
"dropped_frames"	客户端缓存中被覆盖丢弃的帧数。
"drop_ratio"	客户端丢帧比例。
"server_frames_sent"	服务端已发送的帧数。
"server_bytes_sent"	服务端已发送的数据字节数。
"error"	最近一次错误信息。
"last_frame_age"	距离上一帧图像到达的时间，单位为秒。
"uptime"	当前连接已运行的时间，单位为秒。

- 返回值：

返回值	说明
value	查询到的字段值。
None	字段名称不存在或当前没有对应信息。

- 行为说明：
 - `get()` 主要用于调试、状态监控和错误诊断。
 - 字段名称不区分大小写，函数内部会自动转为小写。
 - 如果读取失败，可以通过 `cap.get("error")` 查看最近一次错误信息。
 - 部分字段只有在 `open()` 成功后才有有效值，例如 `"device"`、`"session_id"`。
- 示例：

Python

```
print("编码方式：", cap.get("codec"))
print("图像尺寸：", cap.get("width"), cap.get("height"))
print("服务端相机设备：", cap.get("device"))
print("已解码帧数：", cap.get("decoded_frames"))
print("丢帧比例：", cap.get("drop_ratio"))
```

```
error = cap.get("error")
if error:
    print("相机错误: ", error)
```

3.4.1.10 release()

- 功能：关闭远程相机连接，并释放客户端和服务端相关资源。
- 参数：无。
- 返回值：无。
- 行为说明：
 - `release()` 会通知服务端停止当前相机会话。
 - 服务端收到停止请求后会释放相机资源，包括关闭数据流、停止 V4L2 采集并释放相机占用。
 - 客户端会关闭本地 UDP socket、gRPC 通道以及 HEVC 使用的 ffmpeg 解码进程。
 - 程序结束前应调用该函数，建议放在 `finally` 中，确保异常退出时也能释放资源。
- 示例：

```
Python
cap = RemoteCameraCapture(
    host="192.168.127.10",
    codec="HEVC",
    width=1280,
    height=720,
    fps=60,
)

try:
    cap.open()
    ok, frame = cap.read(timeout=1.0)
    if ok:
        print("收到图像: ", frame.shape)
finally:
    cap.release()
```

3.4.1.11 测试 demo 程序

`test_remote_camera.py` 用于连接远端相机服务，实时接收视频流，并通过 OpenCV 窗口显示画面。

运行过程中，终端会持续打印客户端解码帧率、丢帧数和服务端发送帧数，可用于快速确认相机是否正常出图、视频流是否稳定。

基本用法

默认 HEVC 测试：

```
Bash
python3 client/test_remote_camera.py
```

推荐显式指定参数：

```
Bash
python3 client/test_remote_camera.py \
  --host 192.168.127.10 \
  --port 50088 \
  --codec HEVC \
  --width 1280 \
  --height 720 \
  --fps 60
```

MJPEG 测试：

```
Bash
python3 client/test_remote_camera.py \
  --host 192.168.127.10 \
  --port 50088 \
  --codec MJPG \
  --width 1280 \
  --height 720 \
  --fps 60
```

退出方式

在显示窗口中按：

- `q`
- `Esc`

也可以在终端按 `Ctrl+C`。

参数说明

参数	默认值	说明
<code>--host</code>	192.168.127.10	远端相机服务 IP。
<code>--port</code>	50088	gRPC 控制端口
<code>--codec</code>	HEVC	视频编码，可选 HEVC 或 MJPG。
<code>--width</code>	1920	请求图像宽度。
<code>--height</code>	1080	请求图像高度。
<code>--fps</code>	60	请求帧率。
<code>--read-timeout</code>	1.0	等待新帧的超时时间，单位秒。
<code>--stats-interval</code>	1.0	统计信息打印间隔，单位秒。
<code>--window-name</code>	Remote Camera	显示窗口名称。
<code>--client-ip</code>	空	手动指定客户端 IP。
<code>--udp-port</code>	0	本地 UDP 接收端口，0 表示自动选择。

3.4.2 夹爪运动控制 API

3.4.2.1 创建实例

- 功能：创建实例，连接 gRPC 通讯
- 参数：

参数	类型	默认值	说明
<code>server_address</code>	str	必填	远端服务 IP+端口号。

- 示例：

```
Python
```

```
# 参数: server_address - gRPC CAN 服务地址（根据实际情况修改）
grip = LingkongGrip(server_address="192.168.127.10:55551")
```

3.4.2.2 grip_init() -> bool

- 功能：夹爪电机初始化。
- 返回值：True 初始化成功。
- 示例：

```
Python
if grip.grip_init():
    print("夹爪初始化成功")
else:
    print("夹爪初始化失败")
    exit(1)
```

3.4.2.3 set_torque_limit(torque_limit: int)

- 功能：设置夹持力参数，范围 10~100，90 值对应夹持力约 20N（长时间使用时，建议夹持力参数设置不超过 90）
- 返回值：None
- 示例：

```
Python
# 设置力矩限制（10~100）
grip.set_torque_limit(50)
```

3.4.2.4 set_speed(speed: int)

- 功能：设置速度参数，范围 10~100
- 返回值：None
- 示例：

```
Python
# 设置速度（10~100）
grip.set_speed(50)
```

3.4.2.5 move_to_pos(pos: int)

- 功能：移动到目标位置，范围 0~1000
- 返回值：None

- 示例:

```
Python
# 移动夹爪到目标位置 (0~1000, 0=闭合, 1000=张开)
grip.move_to_pos(500)
```

3.4.2.6 read_pos() -> int

- 功能: 读当前夹爪位置
- 返回值: 夹爪的当前位置, 范围 0~1000
- 示例:

```
Python
# 查询当前位置 (返回值 0~1000)
pos = grip.read_pos()
print(f"当前位置: {pos}")
```

3.4.2.7 read_cur_tempture() -> int

- 功能: 读夹爪电机温度
- 返回值: 夹爪电机当前的温度
- 示例:

```
Python
temp = grip.read_cur_tempture()
print(f"当前温度: {temp}")
```

3.4.2.8 read_torque_limit() -> int

- 功能: 读力矩设置参数
- 返回值: 当前的力矩设置参数, 范围 10~100
- 示例:

```
Python
torque = grip.read_torque_limit()
print(f"力矩参数: {torque}")
```

3.4.2.9 read_speed() -> int

- 功能: 读速度参数
- 返回值: 当前速度参数, 范围 10~100

- 示例:

```
Python
speed = grip.read_speed()
print(f"速度参数: {speed}")
```

3.4.2.10 测试 demo 程序

```
Python
# 导入 SDK
import time
from dm_lingkong_grip_sdk import LingkongGrip

# 1. 创建夹爪实例
# 参数: server_address - gRPC CAN 服务地址（根据实际情况修改）
grip = LingkongGrip(server_address="192.168.127.10:55551")

# 2. 初始化夹爪（会执行夹紧、找零位等动作）
if grip.grip_init():
    print("夹爪初始化成功")
else:
    print("夹爪初始化失败")
    exit(1)

# 3. 设置力矩限制（10~100）,速度(10~100)
grip.set_torque_limit(50)
grip.set_speed(50)

# 4. 移动夹爪到目标位置（0~1000, 0=闭合, 1000=张开）
grip.move_to_pos(500)    # 移动到一半

time.sleep(2.0)
# 5. 查询当前位置（返回值 0~1000）
pos = grip.read_pos()
print(f"当前位置: {pos}")

# 6. 读取当前温度、力矩参数、速度参数
temp = grip.read_cur_tempture()
print(f"当前温度: {temp}")
torque = grip.read_torque_limit()
print(f"力矩参数: {torque}")
speed = grip.read_speed()
```

```
print(f"速度参数: {speed}")
```

```
# 7. 关闭夹爪  
grip.close()
```

3.4.3 视触觉传感器 API

详细使用参考软件包中的视触觉传感器使用文档《**DM-Tac 系列传感器软件平台&开发手册(使用 DM-Flux)**》。

电爪传感器支持通过 `SensorOptions` 中的 `dev_id` 参数指定左、右传感器。

- 左传感器: `dev_id = "left"`
- 右传感器: `dev_id = "right"`

左右传感器的初始化方式一致，仅需根据实际连接的传感器修改 `dev_id`、`remote_addr` 和 `pc_port` 等参数。

4. 注意事项

4.1 夹爪维护与保养

4.1.1 日常清洁

日常清洁频率如表 4-1 所示。

表 4-1 日常清洁

建议最佳清洁频率	需要的工具	需要的零件
每周一次（若使用环境比较脏，每天一次）	干毛巾或纸巾	无

注意

- 夹爪表面不防水，请使用干毛巾或纸巾进行清洁。
- 清洁顶部视觉模块时，请避免使用粗糙工具刮擦镜头表面。
- 在对夹爪进行清洁、检查或维护前，请先关闭机器人和夹爪电源。
- 操作人员应佩戴静电手环等防静电措施，以免损坏电子元件。

4.1.2 定期检查、护理

定期检查、护理频率如表 4-2 所示。

表 4-2 定期检查

检查频率	需要的工具	需要的零件
每月一次	1. M2 内六角螺丝刀 2. M3 内六角螺丝刀	无（除非有损坏需更换）

注意

- 在对夹爪进行任何操作之前，请先关闭机器人和夹爪电源。
- 清洁人员必须佩戴静电手环等防静电措施，以免损坏电子元件。

检查夹爪步骤如下所示：

1. 检查手指运动是否对称、流畅。可通过手指开合测试行程，确认手指能够正常回到初始位置，运行过程中无明显卡顿、抖动等异常现象。
2. 检查视触觉传感器表面、夹爪自身及顶部相机模块是否存在磨损、污渍或碰撞损伤。若视触觉传感器表面磨损严重并影响夹持，请购买感知层并更换；若夹爪或镜头磨损严重并影响设备工作，请联系戴盟机器人技术支持。
3. 夹爪中心齿轮需定期添加润滑油。该操作需要拆卸螺丝并打开顶部结构后进行，操作时请确保设备已断电，并避免异物进入夹爪内部。
4. 检查以太网接口保护罩是否闭合良好，接口内是否有灰尘或异物。未连接网线时，请保持保护罩闭合，避免灰尘进入接口。

4.2 传感器维护与保养

4.2.1 定期检查与清洁

1. 为确保传感器的长期稳定性与可靠性，请定期检查传感器的外观与工作状态。每隔一段时间检查接线、连接端口及外壳是否有损坏或松动现象。
2. 清洁时，使用微湿的软布或海绵擦拭传感器表面。避免使用粗糙的清洁工具，以防刮伤传感器表面。
3. 在清洁时，避免直接使用酒精、丙酮等有机溶剂，这些可能对硅胶或塑料部件造成损害。推荐使用中性清洁剂。

4.2.2 温度与湿度控制

1. 存放传感器时，请保持环境温度在 -10°C 至 50°C 之间，湿度应控制在相对湿度 30% 至 90% 范围内。避免极端的温湿度环境，这些都可能对传感器的精度和使用寿命产生负面影响。
2. 避免将传感器暴露在阳光直射下。

4.2.3 定期功能测试

1. 在每次使用前，建议对传感器进行简短的功能测试，确保其正常工作。测试时，可以通过模拟实际操作来检测传感器的响应和数据输出。
2. 如发现任何异常（如响应迟缓、数据不准确等），请立即进行故障排查。

4.2.4 防止污染与灰尘

1. 长期存放传感器时，请确保其存放在干净、干燥的环境中。避免尘土或异物进入传感器的接触端口或敏感区域。
2. 如果传感器长期不使用，建议将其放入包装盒中或使用防尘罩，减少外界灰尘对设备的影响。

4.2.5 避免物理损伤

1. 在搬运、存放或使用过程中，请避免对传感器施加过大的外力或撞击。剧烈的冲击可能导致传感器功能失常。
2. 使用时，请确保安装牢固，避免任何不当的压力集中在传感器的感知表面。

4.2.6 线束的更换和感知层的更换

本说明书不涉及传感器线束及感知层的拆卸、更换或维护操作。

有关 DM-Tac 系列传感器线束连接、感知层模块更换，请参阅《**DM-Tac 系列传感器硬件说明书**》。